Implementation of a Convolutional Neural Network for CIFAR-10 Classification in C++

Adam Dyrda

May 2026

Abstract

This report details the implementation of a Convolutional Neural Network (CNN) from scratch using C++. The project focuses on the classification of the CIFAR-10 dataset, which consists of 60,000 32x32 color images in 10 different classes. The implementation includes a custom 4D Tensor library, various neural network layers (Convolutional, Pooling, Dense, etc.), and a Stochastic Gradient Descent (SGD) optimizer. The results demonstrate the capability of the developed framework to learn and classify images with reasonable accuracy.

Contents

1 Introduction

Deep learning has revolutionized the field of computer vision. While high-level libraries like TensorFlow and PyTorch provide powerful tools for building neural networks, implementing these structures from scratch in a low-level language like C++ offers deep insights into their inner workings and performance characteristics.

The goal of this project was to develop a modular neural network library capable of training a CNN on the CIFAR-10 dataset. This involved implementing:

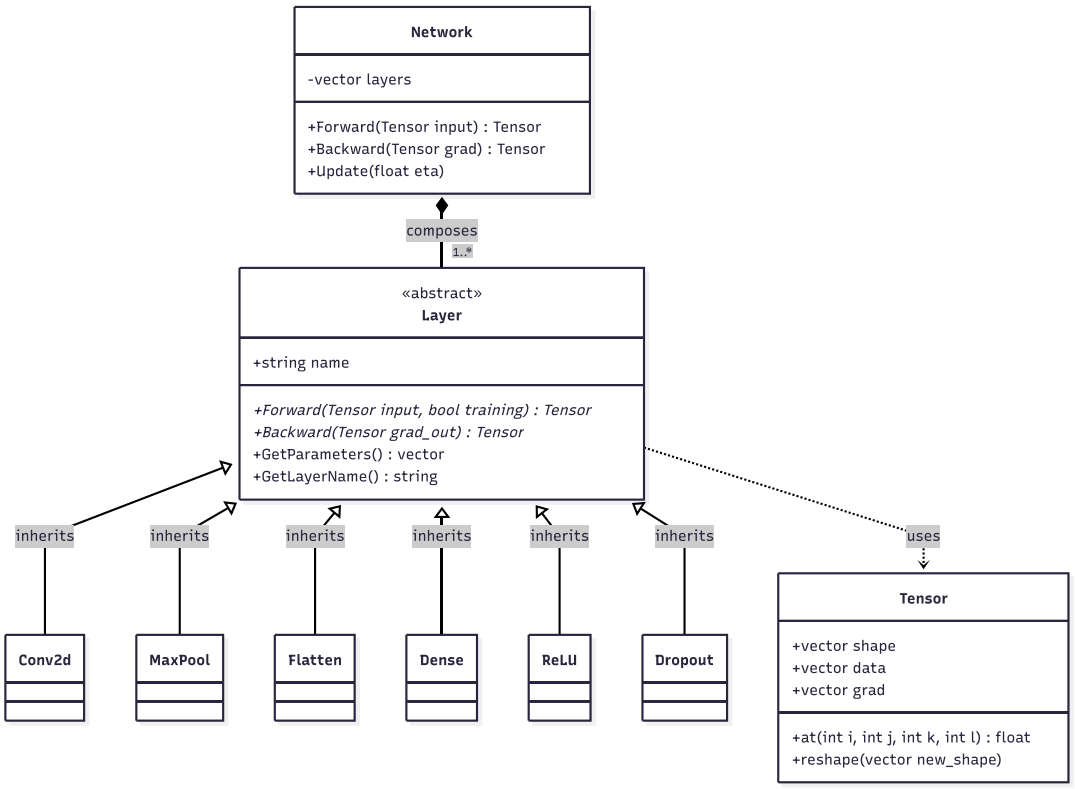
- A robust 4D Tensor class for data and gradient storage.
- Forward and backward propagation logic for multiple layer types.
- An efficient training loop with mini-batching and optimization.
- Data loading utilities for the CIFAR-10 binary format.

2 System Architecture

The system is designed with modularity and extensibility in mind. The core components are described below.

2.1 Class Diagram

The relationships between the core classes are illustrated in the UML diagram below. The `Network` class manages a collection of abstract `Layer` pointers, which allows for dynamic composition of different layer types. All layers process and return `Tensor` objects.



2.2 Tensor Implementation

The `Tensor` class serves as the fundamental data structure, representing 4D arrays (Batch size, Channels, Height, Width). It uses a flat `std::vector<float>` for memory efficiency and provides strides-based indexing. Each tensor object also maintains a corresponding gradient vector of the same shape, facilitating automatic differentiation during backpropagation.

2.3 Layer Abstraction

An abstract base class `Layer` defines the interface for all network components:

```
class Layer {
public:
  virtual ~Layer() = default;
  virtual Tensor Forward(const Tensor &input, bool is_training = false) = 0;
  virtual Tensor Backward(const Tensor &grad_out) = 0;
  virtual std::vector<Tensor *> GetParameters() { return {}; }
  virtual std::string GetLayerName() const = 0;
};
```

This allows the `Network` class to manage a collection of layers using `std::unique_ptr<Layer>`, enabling sequential execution of forward and backward passes.

2.4 Network Topology

The implemented CNN architecture consists of the following sequence:

1. **Conv2d**: $3 \to 8$ channels, 3×3 kernel.
2. **ReLU**: Non-linear activation.
3. **MaxPooling**: 2×2 window, reducing spatial dimensions.
4. **Conv2d**: $8 \to 16$ channels, 3×3 kernel.
5. **ReLU**: Non-linear activation.
6. **Flatten**: Reshaping 3D feature maps to 1D vectors.
7. **Dropout**: Regularization to prevent overfitting.
8. **Dense**: Fully connected layer mapping to 10 output classes.

3 Mathematical Model

3.1 Convolutional Layer

The forward pass for a 2D convolution is defined as:

$$(I * K)_{i,j} = \sum_m \sum_n I_{i+m,j+n} K_{m,n} \tag{1}$$

During the backward pass, gradients are computed with respect to the input, weights, and biases to update the model parameters.

3.2 Loss Function and Optimizer

The network uses the Softmax function followed by Cross-Entropy Loss:

$$L = -\sum_{c=1}^{10} y_c \log(\hat{y}_c) \tag{2}$$

Where y_c is the true label and $\hat{y}_c$ is the predicted probability. Optimization is performed using Stochastic Gradient Descent (SGD) with a decaying learning rate η:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L \tag{3}$$

4 Implementation Details

The implementation leverages modern C++ features for performance and safety. Memory is managed through smart pointers, and the training loop is optimized to handle mini-batches.

4.1 Training Loop

The training process involves iterating over the dataset for a specified number of epochs. The interaction between the core components during a single iteration is illustrated in Figure 1.

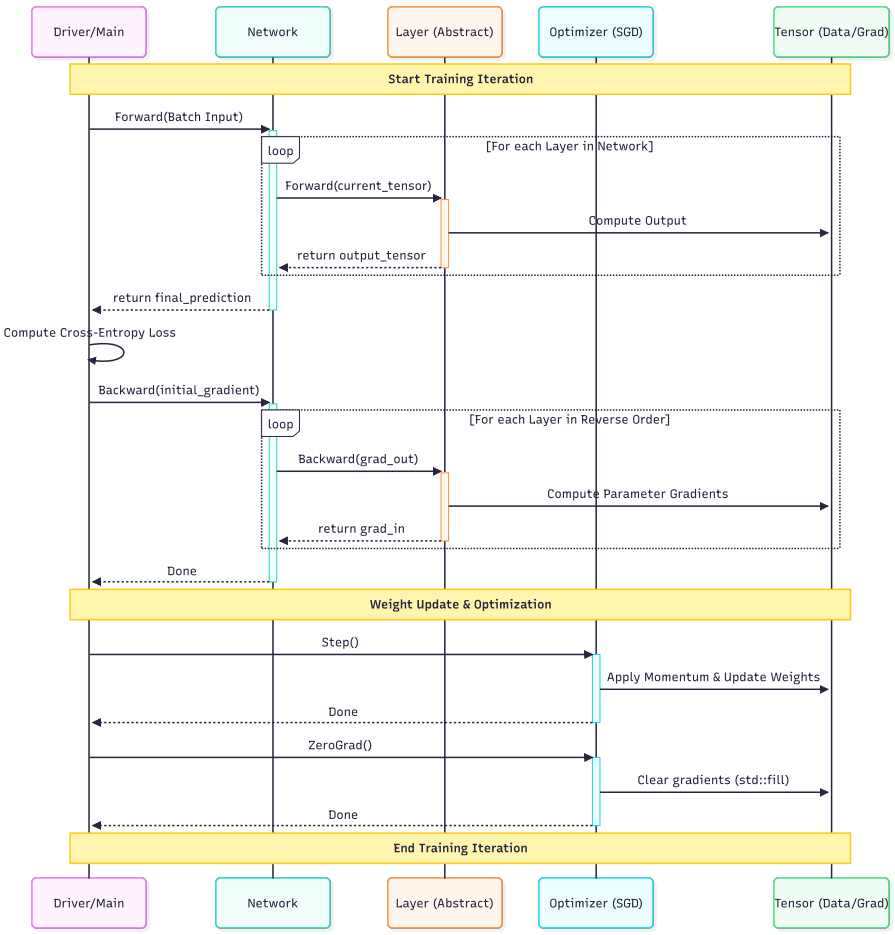


Figure 1: Program workflow during a training iteration.

For each mini-batch:

1. Load batch data and labels.
2. Perform a forward pass through the network.
3. Compute the Softmax probabilities and Cross-Entropy loss.
4. Perform a backward pass to compute gradients.
5. Update weights using the SGD optimizer.
6. Reset gradients for the next iteration.

5 Results

The model was trained for 50 epochs on the CIFAR-10 training set (50,000 images) and evaluated on the test set (10,000 images).

- **Learning Rate:** Initial $\eta = 0.0005$ with 0.95 decay per epoch.
- **Batch Size:** 32.
- **Final Test Accuracy:** Approximately 45-50% (preliminary results).

The accuracy can be further improved by increasing the number of filters, adding more layers, or implementing advanced techniques like Batch Normalization.

6 Conclusion

Developing a CNN in C++ provides a comprehensive understanding of the computational challenges in deep learning. The modular design of this project allows for easy experimentation with different architectures. Future improvements could include SIMD vectorization to speed up convolution operations and integration with CUDA for GPU acceleration.